

LSTM

Основы обработки текстов





Общая постановка задачи обработки последовательностей

Задача обработки последовательностей заключается в анализе и обработке временных рядов или упорядоченных наборов элементов (например, текста, речи, временных сигналов). Основная особенность последовательности — наличие зависимости между элементами, причем порядок следования элементов важен для понимания смысла всей последовательности.

Типичные задачи обработки последовательностей включают:

- Классификация последовательностей: определение категории всего набора данных (например, классификация текста на положительные/негативные отзывы).
- Генерация последовательностей: создание новых элементов последовательности (например, предсказание следующего слова в предложении).
- Последовательность-последовательность («seq2seq») преобразования: преобразование одной последовательности в другую (например, машинный перевод).
- Прогнозирование временного ряда: предсказание будущих значений переменной (например, прогноз цен акций).
- Детектирование аномалий: выявление необычных изменений в данных (например, обнаружение мошенничества в транзакциях банков).

Постановка задачи
обработки
последовательностей
с использованием
LSTM

Дано:

Пусть дана последовательность входных векторов $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$, , где каждый вектор $\mathbf{x}_t$ имеет размерность d . Каждый вектор соответствует одному моменту времени ($t=1, \dots, T$), и элементы последовательности взаимосвязаны друг с другом.

Цель:

Необходимо построить модель, способную эффективно обрабатывать такую последовательность и решать одну из вышеуказанных задач (предсказывать следующие значения, классифицировать всю последовательность целиком, извлекать важные признаки и т.п.).

Математическая формулировка задачи:

Обозначим состояние скрытого слоя в момент времени t как h_t , выход модели в этот момент как y_t .

Основная идея LSTM состоит в наличии трёх ключевых компонент:

Забывание предыдущей информации (f_t) — механизм определения, какую информацию забыть.

Запоминание новой информации (i_t) — выбор того, какая информация должна сохраняться.

Вычисление нового состояния ячейки памяти ($\tilde{C}_t$) — обновление содержимого внутренней памяти.

Формальные уравнения LSTM:

Для каждой временной точки t выполняются следующие шаги:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

— коэффициент забывания старой информации (gate forgetting),
 σ — сигмоидная активация.

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

— коэффициент запоминания новой информации (input gate).

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_{\tilde{c}} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{\tilde{c}})$$

— новое содержимое ячейки памяти.

Формальные уравнения LSTM:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

— обновленное значение внутреннего состояния (cell state), где $\odot$ обозначает поэлементное умножение.

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

— выходной слой активации (output gate).

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

— скрытое состояние на выходе.

Здесь $\mathbf{W}$ — весовые матрицы соответствующих слоев, $\mathbf{b}$ — смещения, а σ и $\tanh$ — нелинейные функции активации.

Обучение нейронной сети

Критерии обучения зависят от конкретной задачи:

Для классификации используют перекрестную энтропию или квадратичную ошибку (MSE):

$$\mathcal{L}(\theta) = - \sum_{n=1}^N \sum_{k=1}^K y_k^{(n)} \log p(y_k | \mathbf{x}^{(n)}; \theta),$$

где $y_k^{(n)}$ — метка класса для примера n , $p(y_k)$ — вероятность правильного класса, заданная моделью.

Обучение нейронной сети

Для регрессии часто применяется среднеквадратичная ошибка:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \|\hat{\mathbf{y}}^{(n)} - \mathbf{y}^{(n)}\|^2$$

Оптимизация производится методом обратного распространения ошибок (backpropagation through time — BPTT) совместно с оптимизаторами вроде Adam или RMSProp.

BPTT в LSTM

Аналогия: Управление кораблем через бурные воды

Представьте себе корабль, плывущий по реке с сильным течением и множеством поворотов. Задача капитана корабля — управлять судном таким образом, чтобы избежать столкновений с берегами реки и благополучно достичь конечного пункта назначения.

Капитан получает команды («градиенты»), поступающие с конца маршрута назад к началу. Эти команды помогают скорректировать траекторию движения судна, позволяя учесть последствия предыдущих решений.

При классическом подходе RNN капитан вынужден учитывать всё течение сразу, начиная с самого начала путешествия, вплоть до завершения пути. Это значит, что команда, данная далеко позади, быстро теряет свою значимость и силу влияния («исчезновение градиента»). Или наоборот, отдельные сильные волны способны привести к резким колебаниям курса («взрыв градиента»).

ВРТТ в LSTM

Почему в LSTM проблема решается лучше?

В отличие от простого корабля, представленного классической RNN, судно, оснащённое системой LSTM, напоминает собой современный круизный лайнер с автоматической системой стабилизации:

- Забывающие ворота: Автоматический контроль над балансировкой нагрузки на двигатели позволяет игнорировать ненужные возмущения, происходящие ранее.
- Запоминающие ворота: Капитан сохраняет ключевые решения и моменты на протяжении долгого пути, используя внутреннюю память судна.
- Выходные ворота: Решение относительно текущего маневра принимается с учётом текущих условий и накопленных знаний, минимизируя влияние старых команд.

Таким образом, LSTM обеспечивает плавное движение судна даже в условиях сложных течений, предотвращая ситуации, когда маленькие команды теряются либо становятся слишком мощными.

Ворота

Ворота — это специальные механизмы внутри структуры LSTM, позволяющие контролировать поток информации между различными этапами обработки. Ворота позволяют регулировать степень влияния прошлого опыта на текущее состояние, а также насколько сильно новые данные будут влиять на внутреннее представление сети.

Три основных типа ворот:

Забывающие (forget) ворота (f_t): Они отвечают за управление информацией, хранящейся в ячейке памяти (cell state). Их задача — решить, какая часть информации должна быть удалена из памяти на каждом этапе. Например, забывая устаревшие или менее значимые детали, модель освобождается от лишней нагрузки и концентрируется на важной информации.

Ворота

Запоминающие (input) ворота (i_t): Отвечают за принятие решений о том, какую новую информацию добавить в ячейку памяти. По сути, это своего рода фильтр, пропускающий только полезные и значимые данные.

Выходные (output) ворота (o_t): Управляют тем, какую долю содержимого ячейки памяти передать наружу в качестве выхода на данном шаге. Это важно, поскольку внутренний объем данных в ячейке памяти может содержать много деталей, но нам не обязательно передавать их все в каждое последующее состояние.

Ворота

Каждый вид ворот играет важную роль в эффективном управлении памятью:

- Забывание: Удаляет старые и малополезные данные, защищая от накопления избыточной информации.
- Запоминание: Регулирует включение новой релевантной информации.
- Выход: Контролирует передачу информации на последующие этапы, фильтруя важные сигналы.

Эти механизмы делают работу LSTM стабильнее и эффективнее, особенно при работе с длинными последовательностями данных. Благодаря управлению памятью, такая структура значительно снижает риск потери значимых признаков (проблема исчезающих градиентов) и перенасыщенности шумом (проблема взрывающихся градиентов).



Процесс обучения LSTM: Backpropagation Through Time (BPTT)

Процесс обучения LSTM

Процесс обучения LSTM осуществляется посредством метода, известного как обратный проход ошибки во времени (BPTT). Этот метод похож на стандартный backpropagation, применяемый в обычных нейронных сетях, однако адаптирован специально для последовательных данных.

Основные шаги алгоритма BPTT:

1. Прямой проход (Forward Pass):

- Данные поступают последовательно по каждому шагу t , рассчитываются внутренние состояния, выходы и промежуточные слои.
- Результат сохраняется в форме итогового вывода сети.

2. Расчёт ошибки (Loss Calculation):

- Рассчитывается разница между фактическими и целевыми результатами (используя, например, кросс-энтропию или MSE).

Процесс обучения LSTM

3. Обратный проход (Backward Pass):

- Ошибка распространяется обратно во времени, распределяясь среди всех шагов (слоев).
- Используются производные функции активации и обновления весов.

4. Обновление весов (Weight Update):

- Применяется оптимизатор (например, Adam, RMSProp) для коррекции весов согласно градиентам, найденным при обратном проходе.

Процесс обучения LSTM

Примеры использования этих техник на практике:

- TensorFlow/Keras API: предоставляет удобные инструменты для ограничения градиентов (`tf.clip_by_value`, `optimizer.gradient_clipping`).
- PyTorch: встроенная поддержка нормализации и `clip gradients` доступна прямо в классе оптимайзера.

Настройка гиперпараметров LSTM-модели

Основные гиперпараметры LSTM:

1. Количество слоёв (Number of layers): Количество повторяемых блоков LSTM в вашей модели. Обычно однослойные модели достаточно эффективны, но многослойные конструкции (2–3 слоя) иногда показывают лучшие результаты на сложных задачах.
2. Размер скрытого слоя (Hidden layer size): Размер вектора состояний скрытого слоя влияет на сложность модели и способность уловить тонкие закономерности. Чем больше размер, тем сложнее модель и тем больше ресурсов потребуется для её тренировки. Часто начинают с небольших размеров (~50–100 единиц), постепенно повышая, если производительность низкая.
3. Dropout Rate: Dropout используется для предотвращения переобучения. Вводится случайное отключение некоторых нейронов на тренировочном этапе, улучшая обобщающую способность модели. Значения около 0.2–0.5 обычно являются разумным выбором.

Настройка гиперпараметров LSTM-модели

4. Batch Size: Размер пакета данных, обрабатываемого одновременно. Большие размеры пакетов ускоряют обучение, но требуют больше оперативной памяти. Маленькие пакеты приводят к замедлению, но повышают точность оценки градиентов. Экспериментируйте с разными размерами, чаще выбирают значения $\sim 32-128$.

5. Learning Rate: Скорость обучения определяет, насколько сильно веса меняются на каждом шаге. Высокие значения могут приводить к быстрому обучению, но вызывают скачки в пространстве параметров, низкие — медленно продвигают обучение. Современные методы, такие как Adam, автоматизируют подбор скорости обучения.

6. Регуляризация (Regularization): Добавление штрафных членов (L1, L2 regularizers) уменьшает эффект переобучения путём наложения ограничений на величину весов. Вес регуляризации задаётся вручную и обычно составляет порядка 0.001–0.01.

Настройка гиперпараметров LSTM-модели

7. Инициализация весов (Weights initialization): Способ начальной установки весов существенно влияет на начало обучения. К примеру, Xavier или He initialization улучшают стабильность обучения.
8. Функция активации (Activation function): Тип активации влияет на качество передачи сигнала между слоями. Традиционно используются ReLU, tanh или sigmoid. Tanh и sigmoid хорошо подходят для внутренних частей LSTM-клеток, так как обеспечивают гладкость и устойчивость.
9. Предварительная обработка данных (Data preprocessing): Масштабирование и стандартизация данных важны для достижения хорошей производительности. Часто применяются min-max scaling или z-normalization.

Подбор гиперпараметров:

Существует несколько подходов подбора гиперпараметров:

- Grid Search: полный перебор комбинаций фиксированного множества значений.
- Random Search: случайный выбор параметров из заранее определённого пространства.
- Bayesian Optimization: продвинутые методы поиска оптимального сочетания параметров, основанные на вероятностных моделях.

Советы по выбору начальных значений:

- Начните с небольшого количества слоёв и нейронов.
- Используйте стандартные техники инициализации весов (Xavier, He).
- Определяйте оптимальный batch size экспериментально.
- Начинайте с умеренного уровня dropout (около 0.2).
- Постепенно повышайте уровень сложности модели, наблюдая за производительностью на тестовом наборе.

Выбор оптимального числа слоев и нейронов

Выбор оптимального числа слоев и нейронов в LSTM-моделях — это сложная задача, зависящая от особенностей ваших данных и целей моделирования. Нет универсального рецепта, подходящего для всех случаев, но существуют общие рекомендации и стратегии, которые помогут вам сделать обоснованный выбор.

Выбор оптимального числа слоев и нейронов

Стратегии выбора числа слоев:

Простые задачи:

- Однослойные модели часто оказываются вполне достаточными для большинства практических задач. Такой подход упрощает интерпретацию результатов и ускоряет процесс обучения.

Среднесложные задачи:

- Два-три слоя могут обеспечить значительное улучшение производительности. Глубокая структура позволяет выявлять сложные шаблоны и взаимодействия между признаками.

Сложные задачи:

- Многослойные модели (более трех слоев) полезны для анализа высокоуровневых паттернов и структурных взаимодействий. Однако будьте осторожны: слишком глубокая сеть может привести к увеличению вычислительной сложности и риску переобучения.

Автоматизированный подбор:

- Можно воспользоваться библиотеками автоматического машинного обучения (AutoML), такими как Hyperopt, Optuna или Ray Tune, которые проводят эксперименты и находят наилучшую конфигурацию автоматически.

Выбор оптимального числа слоев и нейронов

Выбор числа нейронов:

Подбор числа нейронов тесно связан с размером вашего датасета и уровнем сложности решаемой задачи. Число нейронов определяет емкость модели и её способность к извлечению значимых признаков.

- Маленькое число нейронов: повышает риск недообучения (underfitting), когда модель недостаточно гибкая, чтобы выявить закономерности в данных.
- Большое число нейронов: увеличивает риск переобучения (overfitting), когда модель становится слишком сложной и приспособливается к особенностям конкретного тренировочного набора.

Выбор оптимального числа слоев и нейронов

Выбор числа нейронов:

Общие рекомендации:

Начальная точка: начинайте с малого числа нейронов (50–100) и оценивайте производительность на тестовых данных.

Экспериментируйте: постепенно увеличивайте число нейронов, отслеживая поведение кривых обучения и проверяя показатели эффективности.

Используйте визуализацию: графики обучения (learning curves) помогут определить оптимальное число нейронов. Избегайте ситуаций переобучения.

Выбор оптимального числа слоев и нейронов

Важные советы:

Всегда следите за соотношением числа нейронов и объема данных. Большое количество нейронов при небольшом объеме данных почти гарантированно приведет к переобучению.

Используйте регуляризацию (dropout, регуляризаторы) для контроля сложности модели.

Проверьте производительность вашей модели на дополнительном валидационном наборе данных, чтобы убедиться, что выбранная архитектура подходит вашим данным.

Примеры аугментации для последовательностей:

- Перестановка слов или символов: Случайное перемешивание соседних слов или символов в тексте, сохраняя общий смысл и структуру.
- Замена синонимами: Замена отдельных слов синонимичными словами, сохранив семантику и грамматическую согласованность.
- Удаление или вставка токенов: Временное исключение или введение незначащих слов (например, артиклей, предлогов) для разнообразия предложений.
- Интерполяция временных рядов: Искусственное расширение временных интервалов путём введения промежуточных точек, созданных на основе линейной или сплайн-интерполяции.
- Масштабирование амплитуд: Для звуковых или иных временных сигналов возможна модификация амплитуды, что создает дополнительное разнообразие в данных.

Примеры аугментации для последовательностей:

- Смешивание последовательностей: Создание гибридных образцов путём объединения двух коротких последовательностей или сегментов данных.
- Искусственный шум: Введение случайного шума в сигнал или последовательность, имитирующего реальные условия измерений или сбора данных.
- Разбиение и объединение: Разделение длинных последовательностей на короткие фрагменты и формирование новых комбинаций фрагментов.
- Имитация сбоев системы: Модификация входных данных путём случайного удаления элементов или замены ими фиктивных маркеров, отражающих потенциальные сбои или пропущенные данные.



Спасибо за внимание!

 Сергей Ткаченко

 SN Tkachenko@kantiana.ru